Multiple table variations

Hello there. Earlier we introduced you to temporary tables. They're a great resource to use during your analysis because they help you keep your SQL code organized and efficient. You learned how to use a WITH clause to create a type of temporary table. Now, we'll get into some other ways you can create temp tables along with the pros and cons they present. That's one of the great things about data analytics. There's almost always more than one way to get your analysis done.

The SELECT INTO statement is a good example of how to get a temp table done. This statement copies data from one table into a new table but it doesn't add the new table to the database. It's useful if you want to make a copy of a table with a specific condition, like a query with a WHERE clause. So far, we've been using BigQuery to show you how SQL works. But BigQuery doesn't currently recognize the SELECT INTO command. Instead, here's an example of how a SELECT INTO statement might look in another RDBMS. In the statement, a new table named Africa Sales is created using the data from the global sales database about the African region.

```
SELECT
       *
INTO
       AfricaSales
FROM
       GlobalSales
WHERE
       Region = "Africa"
```

Using SELECT INTO is a good practice when you want to keep the database uncluttered and you don't need other people using the table. Now, if lots of people will be using the same table, then the CREATE TABLE statement might be the better option. This statement does add the table into the database. If everyone needs access to the Africa Sales table, your query would start with CREATE TABLE, followed by the same SELECT-FROM-WHERE query as in the SELECT INTO statement. In most relational database management systems or RDBMSs, you can add metadata to describe the data that's contained in the table you've created. This can help make the table easier to understand for anyone using it. The CREATE TABLE statement is also useful for tables that are more complex.

```
CREATE TABLE AfricaSales AS
(
SELECT *
FROM GlobalSales
WHERE Region = "Africa"
)
```

For example, if the code's difficult to replicate, then making a temp table in this way means it'll be safe for you to access later. The way you create a temporary table using the WITH clause or a SELECT INTO or CREATE TABLE statement is usually up to you and your needs. The more you work in SQL, the more you might have preferences as well, especially since there's more than one way to create temporary tables. You may also find that you're working in an RDBMS that uses a different syntax. For example, you might need to use a CREATE TEMP TABLE statement instead of CREATE TABLE. Here's some good news. The syntax that you need for each unique RDBMS is usually pretty easy to find with a quick online search.

How to create temporary tables:

- WITH clauses
- SELECT INTO statements
- CREATE TABLE statements
- CREATE TEMP TABLE statements

But no matter how or where you create temporary tables, there isn't much downside to them. It's good to note though that sometimes building a temp table can interrupt your workflow. Again, that will depend on your objectives and your preferences. You can repeat your code over and over instead of making a temp table but that usually leaves your queries less readable and more vulnerable to typos. As you continue exploring the world of data analytics, you'll find that temporary tables are just one of the many resources you'll be able to use. The more you use them, the easier it'll be to navigate that world.

Question

Which of the following are methods for creating a version of a temporary table using SQL? Select all that apply.

- [x] CREATE TABLE statements

Correct
WITH clauses, CREATE TABLE statements, and CREATE TEMP TABLE statements all create temporary tables in queries.

- [ ] WHERE clauses
- [x] WITH clauses

Correct
WITH clauses, CREATE TABLE statements, and CREATE TEMP TABLE statements all create temporary tables in queries.

- [x] CREATE TEMP TABLE statements

Correct
WITH clauses, CREATE TABLE statements, and CREATE TEMP TABLE statements all create temporary tables in queries.